DuocUC | ONLINE



Guía de aprendizaje

Desarrollando microservicios y resiliencia en la nube con Spring Cloud

Exp 3 – Semana 8

Desarrollo Backend III (PBY2203)

Facilitador disciplinar: Ana Karina Villagrán Ibarra

Asesor par: Ignacio Villarroel

Índice

Introducción a la semana....3
Resultado de aprendizaje....4
Conceptos relevantes....4
Preguntas activadoras....5
Actividad....5
Spring Cloud Security....6
Conceptos clave....7
Implementación....9
Preparando la solución para producción....10
Capa de datos....11
Capa de aplicaciones....11
Conteinerización y Cloud....12
Cierre de la semana....18
Referencias....20
Lecturas de la semana....20
Apuntes....21

Introducción a la semana



En esta última semana, consolidarás todo lo aprendido sobre microservicios y resiliencia en la nube con Spring Cloud, integrando seguridad, tolerancia a fallos y escalabilidad en un sistema backend moderno. Aplicarás los conceptos clave adquiridos en semanas anteriores para desarrollar una solución completa, lista para entornos productivos.

Durante este proceso, profundizarás en la implementación de Spring Cloud Security, configurando mecanismos de autenticación y autorización con OAuth 2.0 y JWT, asegurando la protección de los datos y servicios distribuidos. También reforzarás la tolerancia a fallos con Resilience4j, aplicando Circuit Breakers, Retry Policies y Bulkheads para garantizar la estabilidad y disponibilidad de los microservicios ante interrupciones.

Además, trabajarás con mensajería asíncrona mediante Kafka y JMS, diseñando una arquitectura orientada a eventos que optimiza el flujo de datos en sistemas distribuidos. Finalmente, aprenderás sobre conteinerización con Docker y Docker Compose, preparando tu aplicación para un despliegue eficiente y escalable en la nube.

Esta semana representa el cierre de tu aprendizaje, donde integrarás todos los componentes de una arquitectura backend avanzada, segura y resiliente. Al finalizar, estarás preparado para enfrentar los desafíos del desarrollo en la nube y diseñar soluciones altamente escalables, seguras y preparadas para producción.

Resultado de aprendizaje

El estudiante será capaz de:

RA3. Desarrolla componentes backend avanzados utilizando tecnologías cloud que permitan disponibilizar sistemas resistentes a fallos, de acuerdo a las buenas prácticas de la industria.

Indicador de logro:

IL6. Implementa microservicios utilizando Spring Cloud, para configurar servidores y garantizar la correcta integración entre servicios.

IL7. Configura mecanismos de seguridad y tolerancia a fallos utilizando Resilience4j, garantizando la resiliencia del sistema ante fallos.

IL8. Integra componentes asíncronos de mensajería utilizando Kafka o JMS, para implementar una arquitectura orientada a eventos.

IL9. Implementa políticas de seguridad utilizando Spring Cloud Security para asegurar la protección de datos y servicios en entornos distribuidos.

Conceptos relevantes

Docker	Contenedores	Docker Compose
OAuth	Cloud	Token

Preguntas activadoras

- ¿Crees que lo visto en semanas anteriores es la única forma de abordar el uso de Spring Cloud?
- ¿Cómo crees que es un ambiente productivo?

Actividad

Descripción de la actividad

En esta octava semana realizarás una actividad sumativa individual titulada "Desarrollando microservicios y resiliencia en la nube con Spring Cloud". Deberás implementar mecanismos de seguridad con Spring Cloud Security para proteger la infraestructura distribuida, configurar tolerancia a fallos con Resilience4j e integrar un sistema de mensajería asíncrona utilizando Kafka o JMS para habilitar una arquitectura orientada a eventos.

Spring Cloud Security

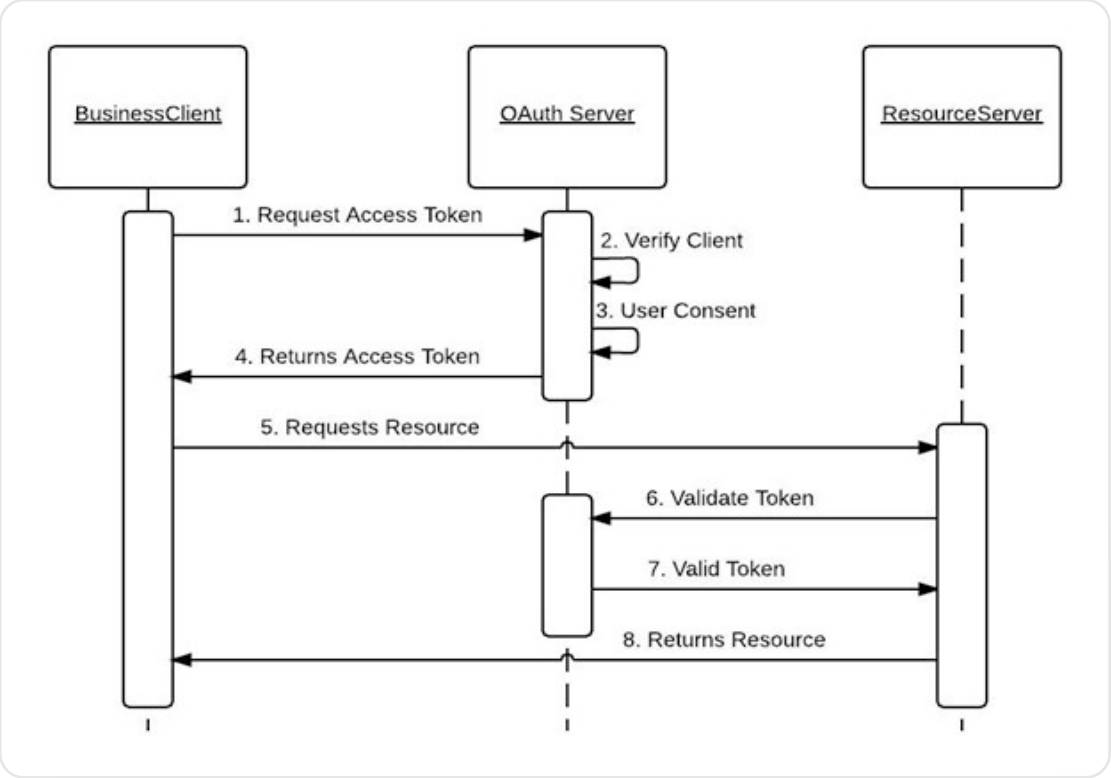
Cuando trabajas en un sistema distribuido basado en microservicios, una de las mayores prioridades es garantizar que cada solicitud que llega a tu sistema sea realizada por usuarios o servicios autorizados. Imagina que tu sistema es como un gran edificio con muchas habitaciones, donde cada habitación representa un microservicio específico. Cada puerta del edificio tiene diferentes requisitos para ser abierta, dependiendo de quién eres y qué necesitas hacer. Por ejemplo, un empleado puede tener acceso a la sala de reuniones, pero no a la bóveda de seguridad.

Ahora, piensa en cómo podrías gestionar este acceso. Podrías crear reglas personalizadas para cada puerta y escribir código específico en cada servicio para manejar estas reglas. Sin embargo, esto se vuelve complejo y difícil de mantener cuando tienes docenas o cientos de microservicios. Por suerte, existen soluciones estándar que resuelven este problema de manera eficiente, como el protocolo OAuth 2.0. Estas soluciones nos ayudan a centralizar la autorización y reutilizar patrones probados en la industria.

Conceptos clave

Figura 1

Diagrama de secuencia de OAuth 2.0



Nota. Esta imagen muestra cómo es la interacción de manera ordenara entre un cliente y un servidor de recursos. Israel, A. (2024). *OAuth2 in Simple Terms.* https://www.propelauth.com/post/oauth2-in-simple-terms

La Figura 1 nos muestra cómo se realiza una interacción cuando hay un Authorization Server disponible implementando OAuth 2.0. Para comprender la figura, te explicamos los conceptos más importantes de la interacción:

- **Servidor de autorización (Authorization Server)**

Es como un guardián centralizado que verifica quién eres y qué permisos tienes. Este servidor genera un token de acceso que actúa como tu pase para moverte por las diferentes habitaciones del edificio (microservicios).

- **Token de Acceso (Access Token)**

Es un "pase" temporal que obtienes del servidor de autorización. Este token contiene información codificada, como tu identidad y los permisos que tienes. Los microservicios validan este token antes de permitirte acceder.

- **Servidor de recursos (Resource Server)**

Representa las puertas del edificio. Cada microservicio actúa como un servidor de recursos que valida el token de acceso antes de permitir que interactúes con los datos o servicios protegidos.

- **Flujo de autorización**

Es el proceso mediante el cual obtienes el token de acceso. Incluye pasos como autenticación del cliente (asegurar que eres quien dices ser) y autorización del usuario (asegurar que tienes permiso para acceder a un recurso).

- **Delegación de Seguridad**

En lugar de que cada microservicio implemente su propia lógica de autenticación, todos delegan esta tarea al servidor de autorización. Esto simplifica el código y mejora la seguridad.

Implementación

Para ver una implementación de un servicio que gestiona identidades en tu proyecto de microservicios, puedes usar el proyecto “Spring Authorization Server”.

Video

Revisa la siguiente grabación del diagrama-secuencia-OAuth2:
https://videosduoc.duoc.cl/media/t/1_0idrz2pw

Links de Interés

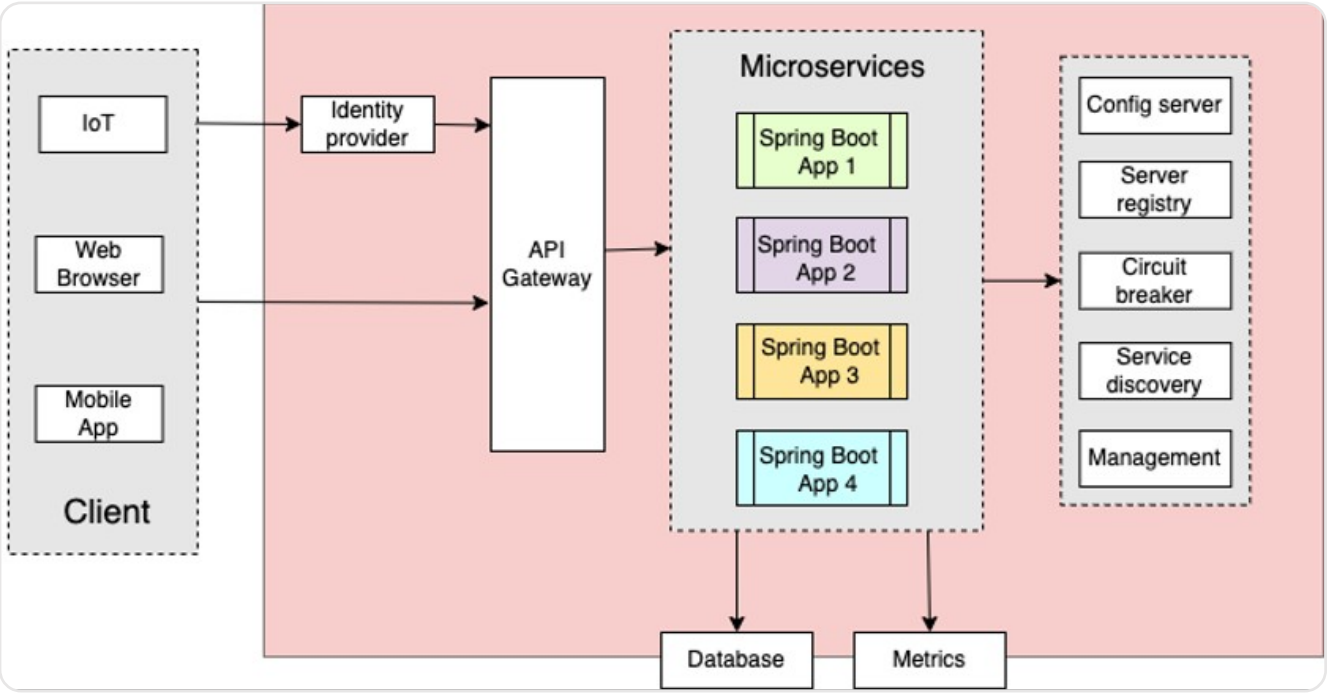
- Paso a paso de implementar OAuth2 con integración de identidad Github
  https://spring.io/guides/tutorials/spring-boot-oauth2
- Ejemplo de código OAuth2
  https://github.com/KariVillagran/spring-security-oauth2

Preparando la solución para producción

Con todo lo aprendido en esta asignatura, ¿crees estar listo/a para liberar tus desarrollos a un ambiente productivo resiliente, tolerante a fallos, seguro y con todos los atributos de calidad deseados en el mundo tecnológico? ¡Es una excelente pregunta! Y para poder responderla abordemos una **Arquitectura de referencia**, la cual mostrará una generalización de una solución para un problema común.

Figura 2

Arquitectura de referencia de soluciones Spring



Nota. Esta imagen muestra un ecosistema Spring considerando todos los elementos abordados en la asignatura. Ranganathan, L. (2023). *Microservices with Spring Boot. Medium*. https://medium.com/@loganathanudt/microservices-with-spring-boot-d2288ef048d5

En la Figura 2, ves cómo integrar todos los conceptos aprendidos en este proceso educativo. Piensa en diseñar una plataforma con varios microservicios, donde cada uno cumple una función específica y todos operan de forma coordinada. Estos microservicios abarcan capacidades clave como seguridad, acceso a datos, resiliencia, reglas de negocio, gestión de servicios y configuraciones, adaptándose a las necesidades de tu proyecto. Con esto en mente, repasemos los aspectos clave que aprendiste.

Capa de datos

Con Spring Batch puedes crear y migrar información de sistemas legacy hacia una plataforma tecnológica moderna, que incluso pueda servir como capa de persistencia de las nuevas APIs que desarrollaras o requieran de la información migrada.

Capa de aplicaciones

Si bien en Backend I y II construiste aplicaciones con Spring Boot, durante este bimestre le agregamos:

- Resiliencia con Resilience4j Implementando Circuit Breaker
- Ambientar un Config Server para consumir configuraciones de forma remota por cada microservicio que lo requiera
- La implementación de un Eureka Server (Server registry) para permitir el Service Discovery en tu ecosistema tecnológico.
- La implementación de un Servidor de Autenticación para el manejo de autenticaciones y autorizaciones entre tu capa de clientes y tus servicios Backend.
- La implementación de un BFF para poder interceptar y administrar de forma optimizada las solicitudes de cada uno de tus potenciales clientes. El BFF puede administrar una autenticación básica o integrarse justamente con un Auth Server para mejorar su implementación.
- Además, según las necesidades del negocio, puede que surja la necesidad de comunicación asíncrona. Para resolver este tipo de requerimientos agregamos la integración de Kafka o JMS.

En resumen, todo lo aprendido nos habilita a decir que: **¡Sí, estamos listos para pasar a producción!**

Conteinerización y Cloud

Cuando trabajas en un sistema moderno, necesitas que tus aplicaciones sean portátiles, escalables y fáciles de desplegar. Aquí es donde Docker se convierte en una herramienta esencial. Como ya debería saber, Docker te permite empaquetar tus aplicaciones junto con todas sus dependencias en contenedores, pequeños "paquetes" que aseguran que tu aplicación funcione de la misma manera en cualquier entorno, ya sea en tu computadora, un servidor de desarrollo o un sistema de producción en la nube.

¿Qué es Docker y por qué usarlo?

Docker es una plataforma de conteinerización que simplifica la forma en que desarrollas, despliegas y ejecutas aplicaciones. Con Docker puedes:

- **Asegurar la consistencia entre entornos:** una aplicación en un contenedor incluye todo lo necesario para ejecutarse: código, librerías, variables de entorno y configuraciones. Esto elimina problemas como "en mi máquina funciona".
- **Facilitar el despliegue continuo:** los contenedores son portátiles y ligeros, lo que hace que moverlos entre entornos sea rápido y eficiente.
- **Optimizar recursos:** a diferencia de las máquinas virtuales, los contenedores comparten el kernel del sistema operativo, ocupando menos espacio y cargando más rápido.

Iniciando con Docker

Con Docker trabajarás principalmente con dos conceptos clave: imágenes y contenedores.

- **Imágenes:** son plantillas que incluyen tu aplicación y todo lo que necesita para ejecutarse. Puedes crear tus propias imágenes o usar imágenes predefinidas desde el Docker Hub, una biblioteca pública de imágenes.
- **Contenedores:** son instancias ejecutables de una imagen. Cada contenedor es independiente, pero puede interactuar con otros si lo configuras.

¿Recuerdas el proyecto de la semana 7 sobre Kafka?, este contiene un archivo docker-compose.yml. Ahora te explicamos para qué sirve:

Links de Interés

Proyecto con integración Apache Kafka:
https://github.com/spring-cloud/spring-cloud-stream-samples/blob/main/kafka-batch-sample/docker-compose.yml

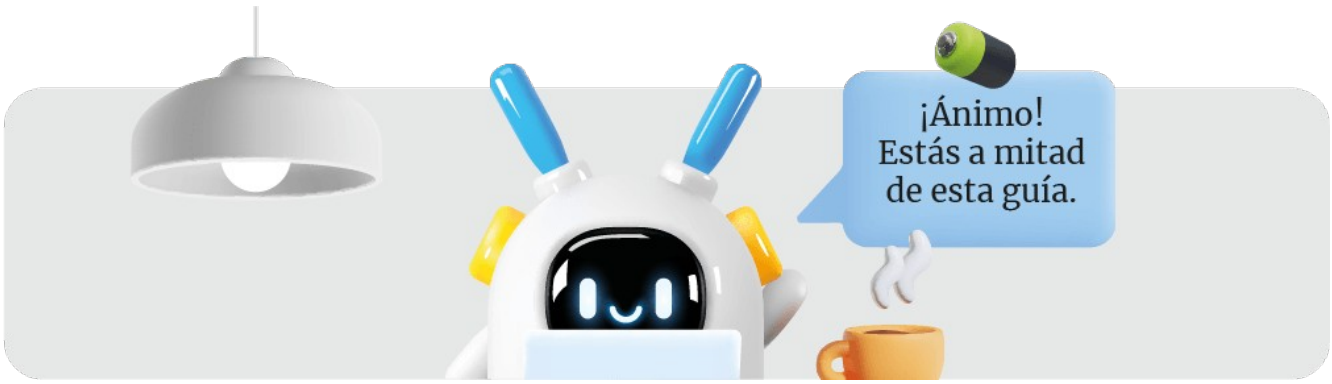
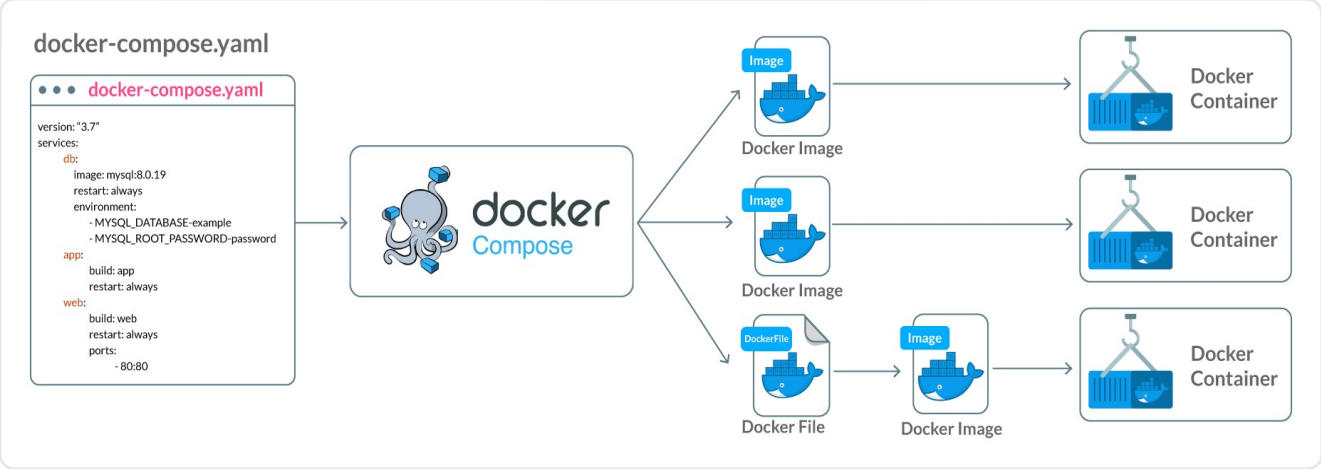


Figura 3

Qué es Docker Compose



Nota. La figura explica cómo desde un archivo YAML puedes crear una aplicación que utiliza diferentes contenedores para su ejecución. Sohail, J. (2024). *What and Why of Docker Compose?* Medium. https://blog.devops.dev/what-and-why-of-docker-compose-dc95314c74b8

Despliegue de contenedores en la nube

Docker no solo facilita el desarrollo local, también simplifica el despliegue en la nube. Con plataformas como AWS, Azure o Google Cloud, puedes orquestar tus contenedores y garantizar que estén disponibles incluso en caso de fallos.

Usando Docker Compose para gestionar múltiples servicios

En sistemas distribuidos, es común que varios servicios trabajen juntos, como un backend, una base de datos, un proxy inverso, servicios de mensajería tipo Kafka, RabbitMQ, entre otras dependencias. Docker Compose te permite definir y gestionar todos estos servicios en un solo archivo docker-compose.yml.

Figura 4

Archivo Docker Compose

```yaml
version: '3'
services:
  kafka:
    image: wurstmeister/kafka
    ports:
      - "9092:9092"
    environment:
      - KAFKA_ADVERTISED_HOST_NAME=127.0.0.1
      - KAFKA_ADVERTISED_PORT=9092
      - KAFKA_ZOOKEEPER_CONNECT=zookeeper:2181
    depends_on:
      - zookeeper
  zookeeper:
    image: wurstmeister/zookeeper
    ports:
      - "2181:2181"
    environment:
      - KAFKA_ADVERTISED_HOST_NAME=zookeeper
```

Nota. Esta imagen muestra una configuración de múltiples herramientas para poder trabajar con Kafka en un ambiente local. Microsoft. (2024). *Visual Studio Code.* (1.95). [Software]. https://code.visualstudio.com/

En la lectura complementaria, encontraras cómo conteinerizar/paquetizar tu aplicación Spring Boot. La idea es que puedas construir una imagen que puedas utilizar en un archivo docker-compose.yml para definir cómo se orquestará tu proyecto.

Una vez logres armar una imagen para configurarla en tu Compose, ya podrás ejecutar el comando: docker-compose up y ¡ver como ocurre la magia!

Figura 5

Archivo Docker Compose con un desarrollo propio

```yaml
version: '3'
services:
  kafka:
    image: wurstmeister/kafka
    ports:
      - "9092:9092"
    environment:
      - KAFKA_ADVERTISED_HOST_NAME=127.0.0.1
      - KAFKA_ADVERTISED_PORT=9092
      - KAFKA_ZOOKEEPER_CONNECT=zookeeper:2181
    depends_on:
      - zookeeper

  zookeeper:
    image: wurstmeister/zookeeper
    ports:
      - "2181:2181"
    environment:
      - KAFKA_ADVERTISED_HOST_NAME=zookeeper

  springboot-app:
    image: your-dockerhub-username/springboot-app:latest
    build:
      context: ./path-to-your-springboot-app
      dockerfile: Dockerfile
    ports:
      - "8080:8080"
    environment:
      - SPRING_DATASOURCE_URL=jdbc:mysql://your-database:3306/yourdb
      - SPRING_KAFKA_BOOTSTRAP_SERVERS=kafka:9092
    depends_on:
      - kafka
```

Nota. Esta imagen muestra una orquestación completa de un backend y sus dependencias.

Microsoft. (2024). *Visual Studio Code.* (1.95). [Software]. https://code.visualstudio.com/

Explorando resiliencia en entornos distribuidos con Docker

La resiliencia es clave en sistemas distribuidos. Docker te permite implementar estrategias como:

- **Reinicio automático:** configura tus contenedores para que se reinicien automáticamente en caso de fallo.
- **Balanceo de carga entre contenedores.**
- **Escalado horizontal:** lanza múltiples contenedores de un servicio para manejar más usuarios o tareas.

Preparación para producción con Docker

Antes de llevar tus contenedores a producción, como una buena práctica, asegúrate de:

1. **Optimizar tus imágenes:** usa comandos como ***docker image prune*** para limpiar imágenes no utilizadas.
2. **Implementar monitoreo:** usa herramientas para supervisar tus contenedores.
3. **Habilitar copias de seguridad:** configura respaldos automáticos para los datos que manejan tus contenedores. Esto lo puedes hacer directamente en cada motor de base de datos o plataforma de datos que utilices.

Al dominar Docker, no solo estarás aprendiendo una herramienta, sino también adoptando un enfoque moderno para construir sistemas escalables y resilientes. Este conocimiento te permitirá destacar en el desarrollo de soluciones tecnológicas preparadas para los retos del mundo actual. ¡Es hora de comenzar a conteinerizarlas y llevarlas a la nube!

Cierre de la semana



¡Felicidades! Has llegado al final de esta octava semana, consolidando tus conocimientos en el desarrollo de microservicios y sistemas resilientes en la nube utilizando Spring Cloud. A lo largo de estos días, no solo exploraste conceptos fundamentales para la creación de sistemas modernos, sino que también los aplicaste a través de prácticas concretas que fortalecen tu capacidad para diseñar soluciones backend avanzadas.

Entre los logros de esta semana, destaca tu comprensión y aplicación de:

- Microservicios con Spring Cloud, donde configuraste servidores que centralizan y facilitan la gestión de servicios distribuidos.
- Tolerancia a fallos, aprendiendo a implementar Circuit Breakers con Resilience4j, garantizando la continuidad de tus servicios incluso ante interrupciones inesperadas.
- Mensajería asíncrona con Kafka o JMS, logrando una arquitectura orientada a eventos que permite un procesamiento eficiente y una comunicación efectiva entre componentes distribuidos.
- Seguridad en sistemas distribuidos, integrando Spring Cloud Security para proteger datos y servicios en la nube mediante la delegación y centralización de las autorizaciones.

Estas habilidades te posicionan como un profesional capaz de enfrentar retos reales en el ámbito del desarrollo cloud, desde la creación de arquitecturas escalables y seguras, hasta el manejo eficiente de entornos distribuidos y resilientes.

Referencias

- Docker Docs. (s.f). *Docker Compose*. Docker. https://docs.docker.com/compose/
- Docker Docs. (s.f). *Engine*. Docker. https://docs.docker.com/engine/
- Israel, A. (2024). *OAuth2 in Simple Terms*. https://www.propelauth.com/post/oauth2-in-simple-terms
- Ranganathan, L. (2023). *Microservices with Spring Boot.* Medium. https://medium.com/@loganathanudt/microservices-with-spring-boot-d2288ef048d5
- Sohail, J. (2024). *What and Why of Docker Compose?* Medium. https://blog.devops.dev/what-and-why-of-docker-compose-dc95314c74b8
- Spring Batch. (s.f.). *Authorization Server.* Spring. https://docs.spring.io/spring-authorization-server/reference/getting-started.html
- Spring Batch. (s.f.). *Spring Projects.* Spring. https://spring.io/projects

Lecturas de la semana

- Spring Batch. (s.f.). *Spring Boot with Docker.* Spring. https://spring.io/guides/gs/spring-boot-docker
- Docker Docs. (s.f.*). Docker Compose.* Docker. https://docs.docker.com/compose/

Apuntes



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.